

Validatie-run protocol

stairlift live 3D telemetry

Nout Mulder • DeVi Comfort

Run-info (invullen vóór je begint):

Datum: _____ Tijd start: _____ Tijd eind: _____

Locatie / klant: _____

Lift serial / ID: _____ Software-versie: _____

Tester: _____ Begeleider aanwezig? ☐ ja ☐ nee

Overzicht

Eén calibratie-run, één rail, vier metingen + twee observaties. Doel: elk DV1–DV3 criterium in de validatie-tabel van het verslag van een echt cijfer voorzien in plaats van [meet].

ID	Meting / observatie	Voor
T1	Bluetooth round-trip tijd (run-gemiddelde, ms)	DV1(b)
T2	Raw encoder jitter, stationair, 10 s (pulses)	DV3(b) baseline
T3	Kalman-filtered jitter, stationair, 10 s	DV3(b)
T4	Positie-error bij referentiepunten (mm)	DV2(b)
O1	Visuele smoothness (geen jumps, oscillation, snapping)	observation
O2	Anomalieën (BT drops, checksum errors, glitches)	observation

Pass/fail logica:

DV1(b) pass: $T1 \leq 100 \text{ ms}$ **DV2(b)** pass: $\max(T4) \leq 5 \text{ mm op rail} \leq 5 \text{ m}; \leq 10 \text{ mm op langere rail}$ **DV3(b)** pass: $T2 / T3 \geq 10$ (factor tien jitter-reductie)

1 Materiaal-checklist

- ☐ Toegang tot één werkende fysieke stairlift
- ☐ Meetlint (rolmaat) met mm-graduatie, minimaal 5 m lang
- ☐ Schilderstape of stickertjes voor 4–5 markers
- ☐ Laptop met de app, Bluetooth-adapter werkend
- ☐ Telefoon voor foto's van het scherm bij elke marker
- ☐ Pen + dit document uitgeprint
- ☐ Reservebatterijen / oplader voor laptop
- ☐ Tijd: ~1 uur op de lift

2 Code-instrumentatie (vóór de run, op kantoor)

Drie kleine toevoegingen in de Godot-codebase. Test ze eerst in demo-mode voordat je naar de lift gaat.

2.1 Logfile redirect

Start de app vanuit terminal zodat alle `print()` output naar een bestand gaat:

```
cd ~/MergeIPC/GD-UP-Studio
godot --path . > ~/logs/run_$(date +%Y%m%d_%H%M).txt 2>&1
```

Bestandsnaam van jouw logfile: _____

2.2 Per-frame CSV-log (voor T2, T3)

In `assets/code/ipc/ipc_lift_controller.gd`, in de functie `on_encoder_received(encoder: int)`, vlak vóór de `_kf_predict(dt)` aanroep:

```
# [VALIDATIE] tijdelijk: per-meting log voor jitter-analyse
print("[CSV] %d,%d,%.6f,%.6f,%.6f" % [
    Time.get_ticks_msec(), encoder,
    _kf_pos, _kf_vel, _display_pos if not is_nan(_display_pos) else 0.0
])
```

Geeft regels: `[CSV] timestamp_ms, raw_encoder, kf_pos, kf_vel, display_pos`.

2.3 Bluetooth round-trip log (voor T1)

In `assets/code/ipc/net/lift/lift_client.gd`, in de functie `send_and_receive(...)`, nét na de while-loop die op de response wacht (rond regel 137, vóór `var result := _pending_result`):

```
if _pending_result != null:
    print("[BT_RTT] msg_id=%d rtt_ms=%.1f" % [msg_id, elapsed_ms])
```

2.4 Marker-knop (voor T4)

Bovenaan `ipc_lift_controller.gd` (class-velden):

```
var _marker_was_pressed: bool = false
```

In `_process(delta)` aan de top:

```
# [VALIDATIE] markeer huidige positie als referentiepunt-passage
if Input.is_key_pressed(KEY_SPACE) and not _marker_was_pressed:
    _marker_was_pressed = true
    print("[MARKER] t=%d encoder=%d kf_pos=%.5f display_pos=%.5f" % [
        Time.get_ticks_msec(), last_encoder_value,
        _kf_pos, _display_pos if not is_nan(_display_pos) else 0.0
    ])
elif not Input.is_key_pressed(KEY_SPACE):
    _marker_was_pressed = false
```

2.5 Sanity-check vóór vertrek

- ☐ Build de app, geen compile errors
- ☐ Run in demo mode: `[CSV]` regels verschijnen continu
- ☐ Druk spatie: `[MARKER]` regel verschijnt
- ☐ Demo mode genereert geen BT, dus `[BT_RTT]` pas op locatie zichtbaar
- ☐ Logfile groeit, output is leesbaar

Sanity-check voltooid op (datum/tijd): _____

3 Op locatie — marker setup

3.1 Stap 0a — Lift uit en rail meten

- ☐ Lift uitgeschakeld, sleutel uit, hoofdschakelaar uit
- ☐ Meet de *totale* rail-lengte met meetlint (van bottom-stootblok tot top-stootblok)
- ☐ Tel het aantal horizontale bends visueel
- ☐ Schat globaal welke segment-types (straight, up-bend, down-bend, h-left, h-right)

Totale rail-lengte: _____ mm

Aantal horizontale bends: _____

Aantal verticale bends: _____

Omschrijving rail (bv. “rechte traploop met één bocht onderaan”):

3.2 Stap 0b — Marker-posities kiezen en meten

Kies 4–5 markers, verspreid over de rail. Plak een stukje tape op de rail bij elke gekozen positie, en meet daarna met de meetlint vanaf het bottom-stootblok hoeveel mm de tape zit. Noteer hieronder.

Tip: zet markers exact daar waar de chair een herkenbaar oppervlak van de rail passeert (bv. een bout, een rail-overgang), zodat je tijdens de run goed kunt zien wanneer de chair de marker bereikt.

ID	Beschrijving (waar zit de marker?)	Fysieke positie (mm)
MP1		
MP2		
MP3		
MP4		
MP5		

4 De run zelf

4.1 Stap 0c — Opstart en verbinding (~10 min)

Wat je doet: laptop aan, terminal openen, app starten met log redirect, en controleren of telemetrie binnenkomt.

- ☐ Laptop aan, op een vlakke ondergrond binnen handbereik
- ☐ Terminal openen, commando uit sectie 3.1 typen (niet uitvoeren)
- ☐ Lift inschakelen, sleutel aan
- ☐ Druk *enter* op terminal — de app start, logfile begint te schrijven
- ☐ In de app: ga naar het IPC-scherm met de ghost chair view

- ☐ Verbind via Bluetooth met de lift
- ☐ Wacht tot in de console regels verschijnen met [CSV] en [BT_RTT]

Klok-tijd app gestart: _____

Klok-tijd BT verbonden: _____

Probleem-oplossing: Geen [CSV] regels? Controleer of de code-aanpassing uit §3.2 is doorgevoerd. Geen [BT_RTT]? Controleer §3.3. App crash? Stop, herstart, noteer de tijd en het type fout hieronder.

Eventuele opstart-issues:

4.2 Stap 1 — Stationaire baseline (10 s — voor T2 en T3)

Wat je doet: chair staat helemaal stil, je laat 10 seconden data lopen. Tussendoor druk je twee keer spatie om begin/eind in de logfile te markeren.

- ☐ Chair op MP1 (bottom-marker), motoren uit, chair geheel stil
- ☐ Wacht ~5 s zodat eventuele opstartfluctuaties verdwijnen
- ☐ **Klik spatie** in de terminal-window (lijn [MARKER] verschijnt). Dit is begin baseline.
- ☐ Noteer hieronder onmiddellijk de klok-tijd
- ☐ Wacht 10 seconden (gebruik telefoon-timer)
- ☐ **Klik spatie** opnieuw. Dit is eind baseline.
- ☐ Noteer eind-klok-tijd

Baseline start (klok-tijd, hh:mm:ss): _____

Baseline eind (klok-tijd, hh:mm:ss): _____

Encoder-waarde tijdens baseline (van scherm aflezen): _____ pulses

Visuele observatie tijdens baseline (was er zichtbare beweging op het scherm?):

4.3 Stap 2 — Bottom → Top run

Wat je doet: de lift rijdt langzaam van bottom naar top. Bij elke marker druk je spatie + maak je foto + noteer je in de tabel hieronder de getoonde positie.

- ☐ Joystick / startknop op de lift indrukken
- ☐ Lift begint te bewegen, vermoedelijk langzaam (firmware speed 20–50)

Klok-tijd run B→T gestart: _____

Bij elke marker (MP1 t/m MP5) doe je:

1. **Kijken** naar de fysieke chair en de markers

2. **Op het moment dat de chair fysiek over de marker rijdt:** druk *spatie* op de laptop (genereert [MARKER] log-regel)
3. Maak *foto* van het scherm: zorg dat de getoonde positie zichtbaar is
4. Lees de getoonde positie af (mm of % of normalized 0–1, wat de UI ook toont) en noteer in de tabel hieronder

Tip: druk *nét voor* de marker; menselijke reactietijd is ~200 ms en de chair is traag genoeg dat een tikje vroeg/laat niet veel uitmaakt.

Marker	Klok-tijd	Foto?	Display toont	Observatie (smooth? glitch?)
MP1 (B→T)		☐ ja ☐ nee		
MP2 (B→T)		☐ ja ☐ nee		
MP3 (B→T)		☐ ja ☐ nee		
MP4 (B→T)		☐ ja ☐ nee		
MP5 (B→T)		☐ ja ☐ nee		

☐ Lift bereikt top station

Klok-tijd lift bij top: _____

Globale indruk B→T (smooth motion, geen jumps?):

4.4 Stap 3 — Top → Bottom return

Idem als stap 2, maar omgekeerd. Levert tweede meting per marker.

☐ Joystick activeren in tegengestelde richting

Klok-tijd run T→B gestart: _____

Marker	Klok-tijd	Foto?	Display toont	Observatie
MP5 (T→B)		☐ ja ☐ nee		
MP4 (T→B)		☐ ja ☐ nee		
MP3 (T→B)		☐ ja ☐ nee		
MP2 (T→B)		☐ ja ☐ nee		
MP1 (T→B)		☐ ja ☐ nee		

Klok-tijd lift weer bij bottom: _____

4.5 Stap 4 — Afsluiten

- ☐ Lift uit, sleutel uit
- ☐ App netjes stoppen (Ctrl+C in terminal, geen kill -9)
- ☐ Controleer logfile: open in editor, scroll door, verifieer aanwezigheid van [CSV], [MARKER], [BT_RTT] regels
- ☐ Tape van rail halen
- ☐ Foto's naar laptop overzetten (bij voorkeur direct, anders binnen 24 u)

Aantal [CSV] regels in logfile (ongeveer): _____

Aantal [BT_RTT] regels: _____

Aantal [MARKER] regels (verwacht: $10 \times + 2$ baseline): _____

4.6 Observaties opschrijven (binnen 10 min na de run!)

Geheugen vervaagt snel. Schrijf nu op wat je zag, ook als het allemaal goed ging.

O1 — Visuele smoothness:

O2 — Anomalieën (BT drops, glitches, vreemd gedrag):

5 Post-run analyse (thuis / op kantoor)

5.1 T1 — Gemiddelde BT round-trip

Commando:

```
grep "\[BT_RTT\]" ~/logs/run_*.txt | \
  awk -F"rtt_ms=" '{print $2}' | \
  awk '{sum+=$1; n++} END {printf "T1 = %.1f ms (n=%d samples)\n", sum/n, n}'
```

T1 resultaat: _____ ms **n samples:** _____

DV1(b) pass? ☐ ja (≤ 100 ms) ☐ nee

5.2 T2 + T3 — Encoder jitter, stationaire baseline

Extract de stationaire window uit het CSV-log. Eerst de hele CSV grep'en:

```
grep "\[CSV\]" ~/logs/run_*.txt | sed 's/\[CSV\] //' > ~/logs/csv_data.csv
```

Open `csv_data.csv` in een spreadsheet of in Python. Filter de regels waarvan de timestamp tussen je baseline-start en baseline-eind klok-tijd valt (zie stap 1; convert hh:mm:ss naar ticks_msec). Bereken binnen dat venster:

- $T2 = \max(raw_encoder) - \min(raw_encoder)$
- $T3 = \max(display_pos) - \min(display_pos)$, omgerekend naar pulses

T2 raw jitter (peak-to-peak): _____ pulses (dus $\pm$ _____ pulses)

T3 Kalman jitter (peak-to-peak): _____ pulse-equivalent

Ratio T2/T3: _____ $\times$

DV3(b) pass? ☐ ja (ratio ≥ 10) ☐ nee

5.3 T4 — Positie-error per marker

Voor elke marker: vergelijk fysieke positie (sectie 4) met getoonde positie. Marker-passage log-regels:

```
grep "\[MARKER\]" ~/logs/run_*.txt
```

De `display_pos` in de log is normalized (0–1). Omrekenen naar mm: `display_pos × rail_length_mm`.

Marker	Fysiek (mm)	Display B→T (mm)	Display T→B (mm)	Avg (mm)	Error mm
MP1					
MP2					
MP3					
MP4					
MP5					

T4 max error: _____ mm **T4 gemiddelde error:** _____ mm

Rail-lengte was: _____ mm threshold: ☐ 5 mm (≤ 5 m) ☐ 10 mm (langer)

DV2(b) pass? ☐ ja (max $\leq$ threshold) ☐ nee

6 Verslag invullen

Open `project/main.tex`, ga naar Table tab:validation (rond regel 1100). Vervang [meet] per rij:

Verslag-rij	Bron	In te vullen	Pass/fail tekst
Rail length	sectie 4	_____ mm	—
Number of horizontal bends	sectie 4	_____	—
Position error at end station	MP1 of MP5	_____ mm	DV2(b): ☐ pass ☐ fail
Max position error at ref. points	T4 max	_____ mm	DV2(b): ☐ pass ☐ fail
Encoder jitter (raw, stationary)	T2	$\pm$ _____ pulses	—
Encoder jitter (after Kalman)	T3	_____ pulses	DV3(b): ☐ pass ☐ fail
Bluetooth round-trip time	T1	_____ ms	DV1(b): ☐ pass ☐ fail
Visual smoothness through bends	O1	korte zin	—

Logged events paragraaf (regel ~1126): vul werkelijke observaties O2 in.

Build PDF:

```
cd project && latexmk -pdf main.tex
```

Commit:

```
git add project/main.tex project/main.aux project/main.fdb_latexmk \
    project/main.fls project/main.log project/main.pdf
git commit -m "validate: fill measurement results from physical run"
```

7 Eerlijkheids-regels

- ☐ Bij failure (criterium niet gehaald): noteer de werkelijke waarde, markeer *fail*, en bespreek de oorzaak in de Conclusion van het verslag. Niet “een tweede run doen tot het past”.
- ☐ Eén run is genoeg voor proof-of-concept; N=1 is duidelijk in de Limitations vermeld.
- ☐ Bewaar de raw logfile in `logs/` (niet committen, te groot). Bewaar wel een geanonimiseerde samenvatting als `logs/run_summary.txt`.
- ☐ Niets verzinnen. Als T1 niet gemeten is wegens ontbrekende instrumentatie, rapporteer dat: “BT round-trip not measured in this run; instrumentation required for future validation.”

Afsluitende notities / lessons learned:

Versie van dit protocol: 28 mei 2026. Plak een ingevulde kopie bij het verslag op de map / sla op in logs/.